Travaux Dirigés
Laravel – Ingénierie avancée

TD 31 à TD 50

Niveau avancé, expert et synthèse

Licence 3 SIO – RAZAFINDRAIBE Fabrice

Projet fil rouge

Plateforme SaaS de gestion de produits, stocks, commandes, paiements et notifications.

$$\boxed{Laravel + API + Queues + Cache + Security + Observability}$$

puis :

$$\boxed{Architecture + CI/CD + Docker + Patterns distribués}$$

ESMIA – 2026

Table des matières

Organisation générale **2**

31 Notifications multicanales **3**
31.1 1. Situation . . . 3
31.2 2. Créer une notification . . . 3
31.3 3. Canal mail . . . 3
31.4 4. Canal database . . . 4
31.5 5. Déclencher . . . 4
31.6 6. Notifications non lues . . . 4
31.7 7. Travail autonome . . . 5

32 Storage, uploads et fichiers privés **6**
32.1 1. Cas . . . 6
32.2 2. Validation . . . 6
32.3 3. Stockage . . . 6
32.4 4. Nom généré . . . 6
32.5 5. Téléchargement privé . . . 7
32.6 6. Changement de disque . . . 7
32.7 7. Défi . . . 7

33 Consommer une API externe **8**
33.1 1. Cas . . . 8
33.2 2. Service . . . 8
33.3 3. Appel . . . 8
33.4 4. Tester le statut . . . 9
33.5 5. Gestion des erreurs . . . 9
33.6 6. Retry . . . 9
33.7 7. Travail autonome . . . 9

34 Webhooks entrants et sortants **10**
34.1 1. Exemple . . . 10
34.2 2. Vérifier une signature . . . 10
34.3 3. Traitement rapide . . . 10
34.4 4. Duplication . . . 11
34.5 5. Webhook sortant . . . 11
34.6 6. Analyse . . . 11

35 Résilience face aux services externes **12**
35.1 1. Mauvaise version . . . 12
35.2 2. Version améliorée . . . 12
35.3 3. Catégoriser . . . 12
35.4 4. Fallback . . . 12

36 Imports massifs et traitement par lots **14**
36.1 1. Cas . . . 14
36.2 2. Mauvaise solution . . . 14
36.3 3. Lecture progressive . . . 14
36.4 4. Insert par lot . . . 14
36.5 5. Jobs par chunk . . . 15
36.6 6. Statut d'import . . . 15
36.7 7. Défi . . . 15

37 SQL avancé, index et plan d'exécution **16**
37.1 1. Requête . . . 16
37.2 2. Version Eloquent . . . 16
37.3 3. EXPLAIN . . . 16
37.4 4. Index . . . 17
37.5 5. Comparer . . . 17
37.6 6. Question . . . 17

38 Verrous distribués **18**
38.1 1. Situation . . . 18
38.2 2. Lock . . . 18
38.3 3. Comparaison . . . 18
38.4 4. Travail autonome . . . 19

39 Multi-tenancy et isolation des données **20**
39.1 1. Modèle . . . 20
39.2 2. Migration . . . 20
39.3 3. Scope . . . 20
39.4 4. Policy . . . 20
39.5 5. Test critique . . . 21
39.6 6. Défi . . . 21

40 Architecture modulaire **22**
40.1 1. Architecture classique . . . 22
40.2 2. Organisation par domaine . . . 22
40.3 3. Exercice . . . 22
40.4 4. Règle importante . . . 23
40.5 5. Travail autonome . . . 23

41 Feature Flags et déploiement progressif **24**
41.1 1. Cas . . . 24
41.2 2. Table . . . 24
41.3 3. Service . . . 24
41.4 4. Utilisation . . . 24
41.5 5. Travail autonome . . . 25

42 Observabilité et correlation ID **26**
42.1 1. Correlation ID . . . 26
42.2 2. Log structuré . . . 26
42.3 3. Scénario . . . 26
42.4 4. Analyse . . . 27

43 CI/CD et automatisation de la qualité **28**
43.1 1. Pipeline . . . 28
43.2 2. Exemple conceptuel YAML . . . 28
43.3 3. Règle . . . 29
43.4 4. Travail autonome . . . 29

44 Docker et environnement reproductible **30**
44.1 1. Problème . . . 30
44.2 2. Services . . . 30
44.3 3. Dockerfile simplifié . . . 30
44.4 4. Compose conceptuel . . . 30
44.5 5. Travail autonome . . . 31

45 Audit architectural complet **32**
45.1 1. Application fournie . . . 32
45.2 2. Classification . . . 32
45.3 3. Priorisation . . . 32
45.4 4. Livrable . . . 33

46 Pattern Outbox **34**
46.1 1. Problème . . . 34
46.2 2. Table Outbox . . . 34
46.3 3. Même transaction . . . 34
46.4 4. Publisher . . . 35
46.5 5. Défi . . . 35

47 CQRS simplifié **36**
47.1 1. Commandes . . . 36
47.2 2. Queries . . . 36
47.3 3. Organisation . . . 36
47.4 4. Exercice . . . 36
47.5 5. Question . . . 37

48 Introduction à l'Event Sourcing **38**
48.1 1. Modèle classique . . . 38
48.2 2. Modèle par événements . . . 38
48.3 3. Table . . . 38
48.4 4. Reconstitution . . . 39
48.5 5. Projection . . . 39
48.6 6. Analyse . . . 39

49 Création d'un package Laravel **40**
49.1 1. Sujet . . . 40
49.2 2. Structure . . . 40
49.3 3. Composer . . . 40
49.4 4. Service Provider . . . 40
49.5 5. API du package . . . 41
49.6 6. Travail autonome . . . 41

50 Grand projet intégrateur **42**
50.1 1. Sujet . . . 42
50.2 2. Acteurs . . . 42
50.3 3. Modules . . . 42
50.4 4. Contraintes techniques . . . 42
50.5 5. Architecture attendue . . . 43
50.6 6. Architecture non imposée . . . 44
50.7 7. Scénarios obligatoires . . . 44
50.8 8. Tests obligatoires . . . 44
50.9 9. Documentation . . . 45
50.1010. Démonstration . . . 45

Barème du projet final **46**

Progression complète du module **46**

Questions de synthèse **47**

Conclusion pédagogique **48**

Organisation générale

Cette série prolonge les TD précédents.
Les étudiants sont maintenant supposés maîtriser :

— MVC et routage ;
— Blade ;
— Eloquent ;
— CRUD ;
— validation ;
— authentification ;
— policies ;
— transactions ;
— API REST ;
— tests ;
— middleware ;
— Service Container ;
— Events ;
— Queues ;
— Cache ;
— concurrence ;
— déploiement.

À partir du TD 31, la priorité est donnée à :

— l'intégration ;
— la résilience ;
— l'architecture ;
— l'exploitation ;
— la capacité à justifier un choix technique.

TD	Thème	Niveau
31	Notifications multicanales	Avancé +
32	Storage et fichiers	Avancé +
33	API externe	Très avancé
34	Webhooks	Très avancé
35	Résilience externe	Très avancé
36	Imports massifs	Très avancé
37	SQL et index	Très avancé
38	Verrous distribués	Expert
39	Multi-tenancy	Expert
40	Architecture modulaire	Expert
41	Feature Flags	Expert
42	Observabilité	Expert
43	CI/CD	Expert
44	Docker	Expert
45	Audit architectural	Expert
46	Pattern Outbox	Expert +
47	CQRS simplifié	Expert +
48	Event Sourcing	Expert +
49	Package Laravel	Expert +
50	Projet intégrateur	Synthèse

31. Notifications multicanales

Objectifs

L'étudiant doit pouvoir :

- créer une Notification Laravel ;
- utiliser plusieurs canaux ;
- distinguer notification métier et transport ;
- utiliser mail et database ;
- différer une notification ;
- gérer l'échec d'un canal.

31.1. 1. Situation

Lorsqu'une commande devient :

paid

le client doit recevoir :

- une notification dans l'application ;
- un email ;
- éventuellement un SMS simulé.

31.2. 2. Créer une notification

```
php artisan make:notification OrderPaidNotification
```

Exemple :

```
class OrderPaidNotification
    extends Notification
{
    public function __construct(
        public Order $order
    ) {
    }

    public function via(
        object $notifiable
    ): array {
        return [
            'mail',
            'database',
        ];
    }
}
```

31.3. 3. Canal mail

```
public function toMail(
    object $notifiable
): MailMessage {

    return (new MailMessage)
```

```
        ->subject(
            'Commande payee'
        )
        ->line(
            'Votre commande a ete payee.'
        )
        ->action(
            'Voir la commande',
            route(
                'orders.show',
                $this->order
            )
        );
}
```

31.4. 4. Canal database

```
public function toArray(
    object $notifiable
): array {

    return [
        'order_id' =>
            $this->order->id,

        'message' =>
            'Commande payee',
    ];
}
```

31.5. 5. Déclencher

```
$user->notify(
    new OrderPaidNotification(
        $order
    )
);
```

31.6. 6. Notifications non lues

Afficher :

```
$request->user()
    ->unreadNotifications;
```

Marquer comme lues :

```
$request->user()
    ->unreadNotifications
    ->markAsRead();
```

31.7. 7. Travail autonome

Travail en autonomie

Créer des notifications pour :

- stock faible ;
- commande expédiée ;
- paiement refusé.

Décider pour chacune :

- canal ;
- utilisateur destinataire ;
- synchrone ou queue ;
- importance.

Questions d'analyse

Une erreur d'envoi d'email doit-elle annuler une commande déjà payée ?
Justifier en distinguant transaction métier et effet secondaire.

32. Storage, uploads et fichiers privés

Objectifs

L'étudiant doit pouvoir :

- utiliser l'abstraction Storage ;
- gérer plusieurs disques ;
- valider un fichier ;
- gérer fichiers publics et privés ;
- créer un téléchargement contrôlé ;
- éviter de coupler le métier au système de fichiers.

32.1. 1. Cas

Un produit peut disposer de :

- image ;
- fiche PDF ;
- document technique.

32.2. 2. Validation

```
$data = $request->validate([
    'document' => [
        'required',
        'file',
        'mimes:pdf',
        'max:4096',
    ],
]);
```

32.3. 3. Stockage

```
$path =
    $request
        ->file('document')
        ->store(
            'product-documents'
        );
```

32.4. 4. Nom généré

Éviter de faire confiance au nom fourni par l'utilisateur.
Exemple :

```
$name =
    Str::uuid() . '.pdf';

$path =
    $request
        ->file('document')
        ->storeAs(
            'product-documents',
```

```
            $name
        );
```

32.5. 5. Téléchargement privé

Créer une route autorisée :

```
public function download(
    ProductDocument $document
) {
    Gate::authorize(
        'view',
        $document
    );

    return Storage::download(
        $document->path
    );
}
```

32.6. 6. Changement de disque

Comparer :

```
Storage::disk('local')
```

et :

```
Storage::disk('public')
```

Discuter comment l'application pourrait plus tard utiliser un stockage distant.

32.7. 7. Défi

Défi avancé

Créer un système où :

- image produit = publique ;
- facture = privée ;
- fiche technique = accessible à tout utilisateur authentifié.

Aucune URL privée ne doit contourner les autorisations Laravel.

33. Consommer une API externe

Objectifs

L'étudiant doit pouvoir :

- utiliser Laravel HTTP Client ;
- traiter les statuts HTTP ;
- utiliser timeout et retry ;
- isoler l'accès externe dans un service ;
- transformer une réponse externe ;
- gérer une panne.

33.1. 1. Cas

L'application doit consulter un service externe de livraison.
Exemple attendu :

$$POST \rightarrow /shipping/quote$$

avec :

- poids ;
- destination ;
- valeur.

33.2. 2. Service

Créer :

`ShippingGateway`

puis une implémentation :

`HttpShippingGateway`

33.3. 3. Appel

```
$response =
    Http::timeout(3)
        ->post(
            config(
                'services.shipping.url'
            ) . '/quote',
            [
                'weight' =>
                    $weight,

                'destination' =>
                    $destination,
            ]
        );
```

33.4. 4. Tester le statut

```php
if ($response->successful()) {
    return $response->json();
}
```

33.5. 5. Gestion des erreurs

Traiter séparément :

- 400 ;
- 401 ;
- 404 ;
- 429 ;
- 500 ;
- timeout.

33.6. 6. Retry

```php
$response =
    Http::retry(
        3,
        200
    )
    ->timeout(3)
    ->get($url);
```

33.7. 7. Travail autonome

Travail en autonomie

Construire un service externe simulé capable de retourner volontairement :

- succès ;
- 404 ;
- 429 ;
- 500 ;
- délai supérieur au timeout.

Le service Laravel doit réagir correctement à chaque cas.

34. Webhooks entrants et sortants

Objectifs

L'étudiant doit pouvoir :

- recevoir un webhook ;
- vérifier une signature ;
- empêcher les duplications ;
- différer le traitement ;
- créer un webhook sortant ;
- comprendre les retries.

34.1. 1. Exemple

Le fournisseur de paiement appelle :

```
POST /webhooks/payment
```

Body :

```
{
  "event": "payment.succeeded",
  "reference": "PAY-1001"
}
```

34.2. 2. Vérifier une signature

Pseudo-code :

```
$signature =
    $request->header(
        'X-Signature'
    );

$expected =
    hash_hmac(
        'sha256',
        $request->getContent(),
        config(
            'services.payment.secret'
        )
    );

abort_unless(
    hash_equals(
        $expected,
        $signature
    ),
    403
);
```

34.3. 3. Traitement rapide

Le contrôleur :

- vérifie ;

— enregistre l'événement ;
— dispatch un Job ;
— retourne rapidement 200.

34.4. 4. Duplication

Stocker un identifiant externe unique :

```
$table->string(
    'external_event_id'
)->unique();
```

34.5. 5. Webhook sortant

Lorsque la commande est expédiée, notifier un partenaire.

```
Http::post(
    $partnerUrl,
    [
        'order' =>
            $order->reference,

        'status' =>
            'shipped',
    ]
);
```

34.6. 6. Analyse

Questions d'analyse

Pourquoi un webhook doit-il être considéré comme une entrée non fiable, même s'il provient théoriquement d'un partenaire connu ?

35. Résilience face aux services externes

Objectifs

L'étudiant doit pouvoir :

- définir timeout ;
- définir retry ;
- définir backoff ;
- définir fallback ;
- distinguer erreur transitoire et erreur permanente ;
- proposer une stratégie de résilience.

35.1. 1. Mauvaise version

```
$response =
    Http::get(
        $remoteUrl
    );
```

Aucun timeout, aucun traitement d'erreur.

35.2. 2. Version améliorée

```
$response =
    Http::connectTimeout(2)
        ->timeout(5)
        ->retry(
            3,
            500
        )
        ->get(
            $remoteUrl
        );
```

35.3. 3. Catégoriser

Compléter :

Erreur	Retry ?	Justification
400		
401		
404		
429		
500		
Timeout		

35.4. 4. Fallback

Si le service de livraison est indisponible :

- bloquer ?
- utiliser un tarif estimé ?

— mettre en queue ?
— demander de réessayer ?

Travail en autonomie

Proposer une stratégie pour chacun des services :

— paiement ;
— livraison ;
— email ;
— statistiques.

La stratégie peut être différente selon la criticité.

36. Imports massifs et traitement par lots

Objectifs

L'étudiant doit pouvoir :

- traiter un gros fichier ;
- éviter de charger tout en mémoire ;
- utiliser chunking ;
- utiliser batch et queues ;
- conserver les erreurs ;
- produire un rapport d'import.

36.1. 1. Cas

Importer :

100000

produits depuis un CSV.

36.2. 2. Mauvaise solution

```
$rows =
    file(
        $request->file('csv')
    );

foreach ($rows as $row) {
    // creation
}
```

Identifier les problèmes.

36.3. 3. Lecture progressive

Utiliser une lecture ligne par ligne ou par blocs.
Pour chaque bloc :

- valider ;
- préparer les données ;
- insérer ;
- stocker les erreurs.

36.4. 4. Insert par lot

Comparer :

```
Product::create(...)
```

100000 fois,
avec :

```
Product::insert(
    $batch
);
```

36.5. 5. Jobs par chunk

Architecture :

$$Upload \rightarrow Parser \rightarrow Chunks \rightarrow Jobs \rightarrow Database$$

36.6. 6. Statut d'import

Créer une table :

`imports`

contenant :

- status ;
- total_rows ;
- processed_rows ;
- failed_rows ;
- started_at ;
- finished_at.

36.7. 7. Défi

Défi avancé

Le navigateur doit pouvoir consulter :

`/imports/{import}`

et voir la progression :

37800/100000

sans attendre la fin dans la requête d'upload.

37. SQL avancé, index et plan d'exécution

Objectifs

L'étudiant doit pouvoir :

- relier Eloquent et SQL ;
- interpréter un plan d'exécution simple ;
- identifier un full scan ;
- concevoir un index ;
- mesurer avant/après ;
- éviter la sur-indexation.

37.1. 1. Requête

```
SELECT *
FROM orders
WHERE user_id = 10
AND status = 'paid'
ORDER BY created_at DESC;
```

37.2. 2. Version Eloquent

```
Order::where(
    'user_id',
    10
)
->where(
    'status',
    'paid'
)
->orderByDesc(
    'created_at'
)
->get();
```

37.3. 3. EXPLAIN

Exécuter selon le SGBD :

```
EXPLAIN
SELECT *
FROM orders
WHERE user_id = 10
AND status = 'paid'
ORDER BY created_at DESC;
```

Observer :

- table parcourue ;
- index utilisé ;
- estimation des lignes ;
- éventuel tri supplémentaire.

37.4. 4. Index

Créer une migration avec un index composite adapté.
Exemple :

```php
$table->index([
    'user_id',
    'status',
    'created_at',
]);
```

37.5. 5. Comparer

Mesurer :

— avant ;
— après ;
— petit dataset ;
— gros dataset.

37.6. 6. Question

Questions d'analyse

Pourquoi ne faut-il pas ajouter un index sur toutes les colonnes ?
Analyser le coût en :

— espace ;
— INSERT ;
— UPDATE ;
— maintenance.

38. Verrous distribués

Objectifs

L'étudiant doit pouvoir :

- distinguer lock SQL et lock distribué ;
- protéger une ressource globale ;
- éviter plusieurs exécutions simultanées ;
- utiliser un lock basé sur cache.

38.1. 1. Situation

Trois workers tentent de générer le même rapport journalier.

$$Worker_A$$

$$Worker_B$$

$$Worker_C$$

Une seule génération doit avoir lieu.

38.2. 2. Lock

Exemple :

```php
$lock =
    Cache::lock(
        'daily-report',
        60
    );

if ($lock->get()) {

    try {

        // generer rapport

    } finally {

        $lock->release();
    }
}
```

38.3. 3. Comparaison

Lock SQL	Lock distribué
Protège des lignes DB	Protège une ressource logique
Dans une transaction	Peut couvrir plusieurs services
Lié au SGBD	Souvent lié au cache/Redis

38.4. 4. Travail autonome

Travail en autonomie

Protéger :

- génération du rapport quotidien ;
- import d'un fichier unique ;
- recalcul statistique global.

Choisir entre :

- lock SQL ;
- lock distribué ;
- idempotence.

39. Multi-tenancy et isolation des données

Objectifs

L'étudiant doit pouvoir :

- définir un tenant ;
- empêcher les fuites inter-organisations ;
- utiliser `tenant_id` ;
- utiliser un scope ;
- adapter les policies ;
- tester l'isolation.

39.1. 1. Modèle

$$Organization \rightarrow Users$$

$$Organization \rightarrow Products$$

$$Organization \rightarrow Orders$$

39.2. 2. Migration

Ajouter :

```
$table->foreignId(
    'organization_id'
)->constrained();
```

aux ressources concernées.

39.3. 3. Scope

Exemple conceptuel :

```
Product::where(
    'organization_id',
    auth()->user()
        ->organization_id
);
```

Puis réfléchir à une abstraction permettant de ne pas répéter cette clause partout.

39.4. 4. Policy

Un utilisateur de l'organisation A ne doit jamais modifier un produit de B.

```
return
    $user->organization_id
    ===
    $product->organization_id;
```

39.5. 5. Test critique

Créer :

— Organization A ;
— Organization B ;
— User A ;
— Product B.

Puis vérifier que User A obtient :

403

ou :

404

selon la stratégie choisie.

39.6. 6. Défi

Défi avancé

Créer un test automatisé qui échoue immédiatement si une requête permet à un tenant de lire les données d'un autre.

40. Architecture modulaire

Objectifs

L'étudiant doit pouvoir :

- reconnaître les limites d'une organisation purement technique ;
- organiser par domaine ;
- définir des frontières ;
- éviter les dépendances circulaires ;
- justifier une architecture.

40.1. 1. Architecture classique

```
app/
    Models/
    Services/
    Actions/
    Policies/
    Events/
```

Avec une grande application, les fichiers peuvent devenir difficiles à retrouver.

40.2. 2. Organisation par domaine

Exemple :

```
app/
    Domains/
        Catalog/
        Orders/
        Payments/
        Users/
```

Puis :

```
Orders/
    Actions/
    Data/
    Events/
    Models/
    Policies/
    Services/
```

40.3. 3. Exercice

Classer :

- Product ;
- Order ;
- PaymentGateway ;
- StockAlert ;
- PlaceOrderAction ;
- OrderPolicy ;
- GenerateInvoice.

40.4. 4. Règle importante

Attention

L'objectif n'est pas de déplacer tous les fichiers uniquement pour créer des dossiers.
Une architecture doit résoudre un problème réel de compréhension, de dépendance ou de maintenance.

40.5. 5. Travail autonome

Travail en autonomie

Proposer deux architectures possibles pour le projet.
Pour chacune, discuter :

- avantages ;
- inconvénients ;
- couplage ;
- testabilité ;
- complexité.

41. Feature Flags et déploiement progressif

Objectifs

L'étudiant doit pouvoir :

- expliquer un feature flag ;
- activer une fonctionnalité pour certains utilisateurs ;
- désactiver rapidement une feature ;
- éviter de maintenir trop longtemps deux versions.

41.1. 1. Cas

Un nouveau moteur de remise doit être testé sur 10% des utilisateurs.

41.2. 2. Table

Créer :

`features`

avec :

- key ;
- active ;
- percentage ;
- configuration éventuelle.

41.3. 3. Service

Créer :

`FeatureManager`

avec :

```
public function enabled(
    string $feature,
    ?User $user = null
): bool
```

41.4. 4. Utilisation

```
if (
    $features->enabled(
        'new-discount-engine',
        $user
    )
) {
    // nouvelle logique
} else {
    // ancienne logique
}
```

41.5. 5. Travail autonome

Travail en autonomie

Implémenter :

- activation globale ;
- activation par utilisateur ;
- activation en pourcentage ;
- désactivation immédiate.

42. Observabilité et correlation ID

Objectifs

L'étudiant doit pouvoir :

- distinguer logs, métriques et traces ;
- ajouter un correlation ID ;
- créer des logs structurés ;
- suivre une requête jusqu'à un Job.

42.1. 1. Correlation ID

Créer un middleware.
Pseudo-code :

```
$id =
    $request->header(
        'X-Correlation-ID'
    )
    ?? Str::uuid()->toString();
```

Ajouter l'identifiant :

- dans le log ;
- dans la réponse ;
- dans les Jobs concernés.

42.2. 2. Log structuré

```
Log::info(
    'Order created',
    [
        'correlation_id' =>
            $correlationId,

        'order_id' =>
            $order->id,

        'user_id' =>
            $user->id,
    ]
);
```

42.3. 3. Scénario

Suivre :

$$HTTP \rightarrow OrderService \rightarrow PaymentGateway \rightarrow Job \rightarrow Notification$$

avec le même identifiant.

42.4. 4. Analyse

Questions d'analyse

Quelle différence entre :

- un log ;
- une métrique ;
- une trace ?

Donner un exemple Laravel pour chacun.

43. CI/CD et automatisation de la qualité

Objectifs

L'étudiant doit pouvoir :

- expliquer CI et CD ;
- exécuter les tests automatiquement ;
- bloquer un déploiement si les tests échouent ;
- comprendre une pipeline.

43.1. 1. Pipeline

git push

↓

composer install

↓

Tests

↓

Build

↓

Deploy

43.2. 2. Exemple conceptuel YAML

```
steps:
  - checkout

  - install PHP

  - composer install

  - php artisan test

  - npm install

  - npm run build
```

43.3. 3. Règle

Attention

Le déploiement ne doit pas être exécuté si l'étape de test échoue.

43.4. 4. Travail autonome

Travail en autonomie

Concevoir une pipeline contenant :

1. installation ;
2. cache des dépendances ;
3. tests ;
4. analyse statique éventuelle ;
5. build frontend ;
6. artifact ;
7. déploiement conditionnel.

44. Docker et environnement reproductible

Objectifs

L'étudiant doit pouvoir :

- expliquer un container ;
- distinguer image et container ;
- construire un Dockerfile ;
- relier PHP et base de données ;
- utiliser des volumes ;
- externaliser la configuration.

44.1. 1. Problème

"Chez moi ça marche."

Nous voulons fournir un environnement reproductible.

44.2. 2. Services

Architecture :

```
app
database
redis
queue
```

44.3. 3. Dockerfile simplifié

```
FROM php:8-fpm

WORKDIR /var/www

COPY . .

RUN docker-php-ext-install pdo_mysql
```

À adapter à la version réelle de PHP.

44.4. 4. Compose conceptuel

```
services:

  app:
    build: .
    volumes:
      - .:/var/www

  database:
    image: mysql

  redis:
    image: redis
```

44.5. 5. Travail autonome

Travail en autonomie

Créer un environnement comprenant :

- Laravel ;
- MySQL ou PostgreSQL ;
- Redis ;
- worker queue.

Un nouvel étudiant doit pouvoir lancer le projet avec très peu de configuration locale.

45. Audit architectural complet

Objectifs

L'étudiant doit pouvoir :

- analyser une application inconnue ;
- cartographier ses responsabilités ;
- identifier dette technique ;
- prioriser les corrections ;
- proposer un plan réaliste.

45.1. 1. Application fournie

Elle contient volontairement :

- contrôleurs de 400 lignes ;
- SQL dans Blade ;
- politiques absentes ;
- services circulaires ;
- duplication ;
- cache incohérent ;
- N+1 ;
- Jobs non idempotents ;
- absence de tests.

45.2. 2. Classification

Classer :

Catégorie	Problèmes
Architecture	
Sécurité	
Performance	
Données	
Tests	
Déploiement	

45.3. 3. Priorisation

Pour chaque problème :

- gravité ;
- fréquence ;
- coût de correction ;
- risque de régression.

45.4. 4. Livrable

Livrable attendu

Rapport de 5 à 10 pages :

1. architecture actuelle ;
2. principaux défauts ;
3. risques ;
4. priorités ;
5. architecture cible ;
6. plan de migration.

46. Pattern Outbox

Objectifs

L'étudiant doit pouvoir :

- expliquer la perte d'événement ;
- garantir l'enregistrement de l'événement métier ;
- utiliser une table Outbox ;
- publier en différé.

46.1. 1. Problème

Scénario :

$$Transaction\ DB \rightarrow COMMIT$$

puis :

$$Application\ crash$$

avant :

$$Queue :: dispatch(...)$$

La commande existe, mais l'événement est perdu.

46.2. 2. Table Outbox

Créer :

```
Schema::create(
    'outbox_events',
    function (Blueprint $table) {

        $table->id();

        $table->string('type');

        $table->json('payload');

        $table->timestamp(
            'published_at'
        )->nullable();

        $table->timestamps();
    }
);
```

46.3. 3. Même transaction

Dans la transaction métier :

```
$order = Order::create(...);

OutboxEvent::create([
    'type' =>
        'order.placed',
```

```
    'payload' => [
        'order_id' =>
            $order->id,
    ],
]);
```

46.4. 4. Publisher

Créer une commande ou Job qui sélectionne :

```
OutboxEvent::whereNull(
    'published_at'
)
```

puis publie.

46.5. 5. Défi

Défi avancé

Rendre le publisher lui-même idempotent.
Le même événement ne doit pas provoquer plusieurs effets irréversibles.

47. CQRS simplifié

Objectifs

L'étudiant doit pouvoir :

- distinguer Command et Query ;
- séparer lecture et écriture ;
- comprendre l'intérêt sans surarchitecturer.

47.1. 1. Commandes

Exemples :

- PlaceOrder ;
- CancelOrder ;
- AdjustStock.

Elles modifient l'état.

47.2. 2. Queries

Exemples :

- SearchProducts ;
- GetDashboard ;
- GetOrderDetails.

Elles ne modifient pas l'état.

47.3. 3. Organisation

Exemple :

```
app/
  Commands/
  Queries/
```

Ou au sein des domaines :

```
Orders/
  Commands/
  Queries/
```

47.4. 4. Exercice

Transformer :

```
OrderController
```

en :

- `PlaceOrderCommand` ;
- `CancelOrderCommand` ;
- `GetOrderQuery`.

47.5. 5. Question

Questions d'analyse

Une petite application CRUD a-t-elle nécessairement besoin de CQRS ?
Présenter les coûts et les bénéfices.

48. Introduction à l'Event Sourcing

Objectifs

Ce TD est expérimental.
L'étudiant doit pouvoir :

- distinguer état courant et historique d'événements ;
- reconstruire un état ;
- créer une projection simple ;
- comprendre les coûts de l'Event Sourcing.

48.1. 1. Modèle classique

On stocke :

```
stock = 105
```

48.2. 2. Modèle par événements

On stocke :

- StockInitialized +100 ;
- StockReceived +20 ;
- StockSold -10 ;
- StockSold -5.

L'état courant est :

$$100 + 20 - 10 - 5 = 105$$

48.3. 3. Table

Créer :

```php
Schema::create(
    'stock_events',
    function (Blueprint $table) {

        $table->id();

        $table->foreignId(
            'product_id'
        );

        $table->string('type');

        $table->integer('quantity');

        $table->timestamps();
    }
);
```

48.4. 4. Reconstitution

Créer :

```
public function currentStock(
    Product $product
): int
```

qui reconstruit le stock.

48.5. 5. Projection

Pour accélérer la lecture, maintenir également une projection :

`product_stock_projection`

48.6. 6. Analyse

Questions d'analyse

Quels avantages apporte l'historique complet ?
Quels nouveaux problèmes apparaissent ?
Pourquoi l'Event Sourcing n'est-il pas nécessaire pour toutes les applications ?

49. Création d'un package Laravel

Objectifs

L'étudiant doit pouvoir :

- créer une bibliothèque réutilisable ;
- configurer Composer ;
- utiliser PSR-4 ;
- créer un Service Provider ;
- fournir configuration et migrations ;
- tester indépendamment le package.

49.1. 1. Sujet

Extraire le système :

Stock Alert

dans un package indépendant.

49.2. 2. Structure

```
stock-alert/
    composer.json
    src/
        StockAlertServiceProvider.php
        StockAlertManager.php
    config/
        stock-alert.php
    database/
        migrations/
    tests/
```

49.3. 3. Composer

Extrait conceptuel :

```
{
  "autoload": {
    "psr-4": {
      "Vendor\\StockAlert\\": "src/"
    }
  }
}
```

49.4. 4. Service Provider

```
class StockAlertServiceProvider
    extends ServiceProvider
{
    public function register(): void
    {
        $this->mergeConfigFrom(
            __DIR__
```

```
            . '/../config/stock-alert.php',
            'stock-alert'
        );
    }

    public function boot(): void
    {
        // config, migrations, etc.
    }
}
```

49.5. 5. API du package

Exemple :

```
StockAlert::check(
    $product
);
```

ou :

```
$manager->check(
    $product
);
```

49.6. 6. Travail autonome

Travail en autonomie

Le package doit :

- être configurable ;
- ne pas dépendre de l'application principale ;
- fournir ses propres tests ;
- être installable par Composer dans un projet de démonstration.

50. Grand projet intégrateur

Objectifs

Le TD 50 constitue une synthèse.
Il ne fournit volontairement presque aucun code.
L'étudiant doit mobiliser les compétences acquises dans l'ensemble du module.

50.1. 1. Sujet

Développer :

Plateforme SaaS de gestion de ventes, stocks et commandes

50.2. 2. Acteurs

Le système comporte :

- administrateur plateforme ;
- organisation cliente ;
- gestionnaire ;
- vendeur ;
- client ;
- application mobile ;
- services externes.

50.3. 3. Modules

Minimum :

- organisations ;
- utilisateurs ;
- produits ;
- catégories ;
- stocks ;
- commandes ;
- paiements ;
- notifications ;
- rapports.

50.4. 4. Contraintes techniques

Le projet doit inclure :

1. authentification Web ;
2. authentification API ;
3. rôles et policies ;
4. multi-tenancy ;
5. transactions ;
6. protection contre les problèmes de concurrence ;
7. API REST ;

8. idempotence de la création de commande ;
9. Events et Listeners ;
10. Jobs et Queues ;
11. Notifications ;
12. Cache ;
13. intégration API externe ;
14. Webhook ;
15. tests automatisés ;
16. Scheduler ;
17. logs structurés ;
18. procédure de déploiement.

50.5. 5. Architecture attendue

Une requête typique :

$$Client$$

$$\downarrow$$

$$Middleware$$

$$\downarrow$$

$$Controller$$

$$\downarrow$$

$$FormRequest + Policy$$

$$\downarrow$$

$$Action/Service$$

$$\downarrow$$

$$Domain + Eloquent$$

$$\downarrow$$

$$Database$$

avec éventuellement :

$$Event \rightarrow Listener \rightarrow Job \rightarrow Queue$$

50.6. 6. Architecture non imposée

Attention

L'étudiant ne sera pas évalué sur le fait d'avoir utilisé le maximum de patterns.
Il sera évalué sur :

- la pertinence ;
- la cohérence ;
- la simplicité ;
- la justification.

50.7. 7. Scénarios obligatoires

Le jury doit pouvoir tester :

1. création d'une organisation ;
2. création de produits ;
3. création d'un utilisateur ;
4. connexion ;
5. création d'une commande ;
6. tentative de survente ;
7. deux commandes concurrentes ;
8. retry de la même requête ;
9. paiement externe simulé ;
10. webhook reçu deux fois ;
11. notification ;
12. génération de facture en queue ;
13. consultation API ;
14. tentative d'accès à un autre tenant ;
15. panne simulée d'un service externe.

50.8. 8. Tests obligatoires

Au minimum :

- authentification ;
- autorisation ;
- validation ;
- isolation tenant ;
- transaction ;
- stock ;
- concurrence ;
- idempotence ;
- API ;
- Job ;
- Event ;
- Notification ;
- webhook.

50.9. 9. Documentation

Le projet doit fournir :

— README ;
— architecture générale ;
— modèle de données ;
— instructions d'installation ;
— fichier d'exemple d'environnement ;
— endpoints principaux ;
— stratégie de tests ;
— procédure de déploiement.

50.10. 10. Démonstration

Durée suggérée :

15 à 20 *minutes*

Structure :

1. besoin ;
2. architecture ;
3. démonstration ;
4. incident simulé ;
5. tests ;
6. limites ;
7. améliorations futures.

Barème du projet final

Critère	Points
Analyse du besoin et modélisation	2
Architecture Laravel	2
Qualité des données et migrations	2
Sécurité et isolation	2
Transactions et concurrence	2
API et intégrations	2
Events, queues et notifications	2
Tests automatisés	2
Performance et observabilité	1
Déploiement et documentation	1
Justification technique et soutenance	2
Total	**20**

Progression complète du module

Niveau 1 – Fondamentaux

$$Installation \rightarrow MVC \rightarrow Routing \rightarrow Blade \rightarrow Eloquent$$

Niveau 2 – Intermédiaire

$$CRUD \rightarrow Validation \rightarrow Relations \rightarrow Authentication$$

$$\rightarrow Services \rightarrow Transactions \rightarrow API \rightarrow Tests$$

Niveau 3 – Avancé

$$Middleware \rightarrow Container \rightarrow Events \rightarrow Queues$$

$$\rightarrow Cache \rightarrow Security \rightarrow Concurrency$$

Niveau 4 – Ingénierie avancée

$$Webhooks \rightarrow Resilience \rightarrow Bulk\ Processing$$

$$\rightarrow SQL \rightarrow Distributed\ Locks \rightarrow Multi-tenancy$$

Niveau 5 – Expertise

$$Observability \rightarrow CI/CD \rightarrow Docker$$

$$\rightarrow Outbox \rightarrow CQRS \rightarrow Event\ Sourcing$$

$$\rightarrow Package \rightarrow Architecture\ globale$$

Questions de synthèse

Un étudiant ayant suivi l'ensemble du parcours doit être capable de répondre aux questions suivantes.

1. Pourquoi séparer le code HTTP du code métier ?
2. Quand utiliser un middleware ?
3. Quand utiliser un Service ?
4. Quand utiliser un Event ?
5. Quand utiliser un Job ?
6. Quand utiliser une transaction ?
7. Quand utiliser un lock SQL ?
8. Quand utiliser un lock distribué ?
9. Qu'est-ce que l'idempotence ?
10. Pourquoi les webhooks doivent-ils être vérifiés ?
11. Pourquoi une API externe nécessite-t-elle un timeout ?
12. Quelle différence entre retry et idempotence ?
13. Quel problème résout une queue ?
14. Quel problème résout le cache ?
15. Quel problème crée un cache mal invalidé ?
16. Qu'est-ce qu'un N+1 ?
17. Pourquoi comprendre SQL reste-t-il nécessaire avec Eloquent ?
18. Qu'est-ce qu'un tenant ?
19. Comment empêcher les fuites entre tenants ?
20. Pourquoi utiliser un Feature Flag ?
21. Qu'est-ce qu'un correlation ID ?
22. Pourquoi automatiser les tests dans la CI ?
23. Quel problème Docker cherche-t-il à réduire ?
24. Pourquoi utiliser un Outbox ?
25. Quelle différence entre Command et Query ?
26. Quel est le coût de l'Event Sourcing ?
27. Quand une architecture devient-elle trop complexe ?
28. Comment savoir si un pattern est réellement utile ?

Conclusion pédagogique

Compétence finale recherchée

Le développeur ne doit plus raisonner uniquement en termes de :

« Quelle commande Artisan faut-il lancer ? »

Il doit pouvoir raisonner en termes de :

responsabilité, cohérence, sécurité, concurrence, résilience, performance, testabilité et maintenabilité.

La question principale devient :

« Quelle garantie l'application doit-elle fournir, et quelle architecture permet de la fournir avec le minimum de complexité nécessaire ? »